ECM2414 Software Development

Multithreaded Card Playing Sim

Design Rationale of the System

CardGame is the central simulation class. All simulation initiation logic is contained in its startSim() method. We wrote the logic here rather than in the main() method so that other classes can create an instance of CardGame and run the simulation.

At the start of the simulation, CardGame requests and validates the number of players and the pack file path from the user. We implemented extensive exception handling with very descriptive error messages to help the user provide valid input next time. For repeated inputs, our project handles exceptions by informing the user of what went wrong and then asking for the input again, without quitting the whole simulation. This improves the user experience. We ran an experiment in which an exception would quit the whole simulation, but this led to technical stack traces being presented to the user, which caused confusion and distrust in the system.

We initially wrote the pack validation logic within CardGame, but since it is related to a pack, we moved it to a static method inside the Pack class. We chose a static method because pack file validation is about a pack in general, rather than any specific pack. This improves code organisation and hence maintainability by keeping related logic together under a single identifier. Continuing with the theme of strong exception handling, the exception messages that might occur during pack validation are written to inform the user exactly why the pack file is invalid, rather than simply stating that it is invalid; yet again, this improves user experience and trust in the system.

Initially, we chose to implement a card using a class with one attribute: value, and the appropriate constructor and getter. Since a card's value will never change during the simulation, we declared the value attribute as final. Furthermore, since the value will never change and all threads will only ever read the card's value, we didn't need to implement any thread-safe aspects to the card class or attribute itself. After implementing this, we realised that Java has what's called a record, which is what we have explicitly defined. Therefore, by updating the card to be a record rather than a class, it has implicitly become final, and the Java compiler automatically creates the required constructor and getters. By improving the card in this way, we have improved code maintainability by making the card definition much simpler and easier to understand.

We have implemented decks to use the queue data structure to store cards, a first-in, first-out (FIFO) structure. Instead of implementing a circular queue, which adds unnecessary complexity, we chose to use a simple linear queue and shift all elements one index left when a card is taken from the front (top) of the deck. We chose a linear queue because the added performance penalty from shifting at most 5 cards is negligible. Furthermore, we initially set a deck to hold a max of 4 cards; however, since all decks are filled to the max at the start of the simulation, and players check if they can discard a card to their right deck i.e., the right deck must not be full, this would lead to a deadlock, where each player is continuously unable to discard to its full right deck. Therefore, each deck has a maximum capacity of 5 cards to avoid this deadlocking situation.

Staying with the theme of data structures, we have implemented a pack to use a partial queue structure, similar to a deck. However, since a pack is only going to be used at the very start of the simulation, to add the values read from the input file and to distribute the cards to the players and decks, we only needed to accommodate adding cards until full, then taking all cards, in that order. Therefore, each time a card is taken from the top of the pack, all cards stay put, and a pointer is simply

shifted one index to the right to mark the new top card, and the reference to the old top card is removed. By implementing a pack this way, we reduce unnecessary complexity and improve performance; a shifting method would result in linear time complexity for a pack of size 8n.

The draw-discard operation that a player performs has been made truly atomic by using a deck-wide custom lock and synchronisation. Before a player attempts to perform its take-discard operation, it checks whether its left and right decks are unlocked and whether it can take and discard to its decks, respectively. Only then does it lock its left and right decks and start the atomic take-discard operation. Once the operation has finished, the player unlocks both its decks. Previously, the take and discard methods of a deck were synchronised, ensuring no two threads/players could take or discard a card at the same time. However, this did not prevent one player from manipulating a deck between another player's take and discard operations, rendering the take-discard operation non-atomic. Therefore, by adding the deck-wide lock alongside the synchronised methods, we have achieved a fully atomic take-discard operation.

Furthermore, we realised that this situation could cause a deadlock to occur. Therefore, we improved our deck locking mechanism to utilise resource reordering. At the start of a player's move, it always locks the deck with the lowest value, then the other deck. This way, we ensure the situation where a ring of dependencies might occur, will not occur, preventing a deadlock; further improving the robustness of our simulation.

Initially, the game state logic was interwoven with the player's game logic; we used static volatile variables to keep track of whether a player has won the game and the winning player's value. This way, each player would be updated. However, this method tightly couples game state and player logic, reducing maintainability and making testing more difficult. Therefore, we decoupled the game state logic and created a GameController interface, which is implemented as the PlayerGameController nested class within CardGame. During player creation, a reference to this controller is passed into the constructor and used to declare that they have won and to access game state updates.

At the start of each player's go, they first contact the game controller and ask whether the game is over; i.e., whether a player has won the game. If not, the player then calculates whether it has a winning hand; if so, the player again communicates with the game controller to say it has won the game. Since a player might have already informed the game controller that they have won before this player, the game controller will ignore this second winning player request in this scenario and return false to indicate that this player is not the true winner. However, when the player comes to log which player won the game, it doesn't keep a local record, so it must ask the game controller who won. By decoupling the logic in this way, the first player to declare that they have won will always be the one which is logged to file.

Previously, once a player declared that they had won the game, the CardGame class immediately started to log the final contents of each deck. However, since the take-discard operation is atomic, a player could be in the middle of taking and discarding a card, meaning the final deck contents that are logged may not be the final contents. Therefore, we have implemented a check-out system, where when a player has finished, it tells the game controller that it has exited the game. Once the last player has informed the controller that it has finished and exited, only then does the controller log each deck's contents to their respective log file. By implementing this check-out logic, the system ensures all players have finished their atomic take-discard operations and that the true final contents of the decks are logged.

Design Rationale of Tests

Every class is designated its own test class, with the exception of the Card Record, as getters are meaningless to test and would only bloat test results. All unit tests exist to test meaningful functionality that greatly affect the flow of the program, or to ensure it follows the specification exactly. Every method within classes that face the human in any way are listed as package-private, allowing unit tests to be ran within the package.

Tests are written in JUnit 6 to make use of the most modern methods and features, and each unit test is annotated with a descriptive display name for easier reading in test results. Larger test classes with logical splits of test types are separated with nested classes; these may be for boundary tests, file/logging input and output, game state and behaviour logic, e.t.c. There also exists tests to check the inverse relationships, and test results are cross-checked where applicable. All possible error conditions that could occur are forced to ensure that the rest of the system either survives or stops where appropriate.

Tests were written during development, to help design code better and identify any unforeseen issues.

Each test class has associated local helper methods, and there is a global TestUtils class for shared static helpers across test classes (such as for generating valid pack files).

Conditions to test are based on the specification listings. For specific values/operations, boundary, edge and corner test cases are written where appropriate. Where necessary, setup (@BeforeEach) and cleanup (@AfterEach) methods are used to provide new pack files, or to create/wipe temporary directories for files to be written/read to.

Due to the architecture's interaction layer that splits the client from the game state, GameController, a mock game controller object exists to simulate and force certain scenarios for clear deterministic testing. Instead of just the GameState purely supporting system in and out, the GameController interface that the PlayerGameController and MockGameController inherit from, supports different stream subclasses. Therefore, MockGameController uses a buffered byte stream to simulate user input and also validate the system's returned outputs. This allows us to test, initial prompts, reprompting, error messages and outputs without interacting with the console.

Tests also were performed to ensure that no matter the condition, an object can never be viewed in an inconsistent state, testing atomic behaviour, synchronized operations and locking.

Tests make use of repetition and parameterisation where applicable. PackTest's validPacksForDifferentPlayerCounts() test executes itself multiple times with varying player counts, supporting wide use case testing with clean and maintainable code. Repeated tests are executed on pack files to ensure that despite the non-deterministic behaviour of threads, the program still always behaves as expected.

The CardGameTest file makes use of integration tests to confirm parts of the system work correctly together. It uses repeated tests of both fixed and generated pack files to test game state invariants. Performance tests also exist to ensure that games complete within set times, ensuring that deadlocks or similar issues do not occur.

Development Session Tracking

02/10/26 14:00 3hrs – Driver: Cameron Russell, Navigator: Jake Fordy

05/10/26 13:00 1hr – Driver: Cameron Russell, Navigator: Jake Fordy

05/10/26 18:00 30min – Driver: Cameron Russell, Navigator: Jake Fordy

08/10/26 14:30 3hrs – Driver: Cameron Russell, Navigator: Jake Fordy

12/10/26 10:30 3hrs – Driver: Cameron Russell, Navigator: Jake Fordy

12/10/26 17:30 2.5hrs – Driver: Cameron Russell, Navigator: Jake Fordy

13/10/26 12:30 2hrs – Driver: Cameron Russell, Navigator: Jake Fordy

14/10/26 15:00 3hrs – Driver: Cameron Russell, Navigator: Jake Fordy

15/10/26 14:30 3hrs – Driver: Jake Fordy, Navigator: Cameron Russell

16/10/26 10:30 3hrs – Driver: Jake Fordy, Navigator: Cameron Russell

17/10/26 11:00 5hrs – Driver: Jake Fordy, Navigator: Cameron Russell

18/10/26 12:00 3hrs – Driver: Jake Fordy, Navigator: Cameron Russell

19/10/26 10:00 3hrs – Driver: Jake Fordy, Navigator: Cameron Russell

19/10/26 17:00 6hrs – Driver: Jake Fordy, Navigator: Cameron Russell

20/10/26 09:00 2hrs – Driver: Jake Fordy, Navigator: Cameron Russell

Significant Commits

- **357d776 14/10/26** - "feat: finished implementation of stopAllOtherThreads() and wrote Javadoc for it."
    - First working version of the simulation.
- **ff71577 17/10/26** - "feat: Created GameController interface and implemented in CardGame to control Player's access to methods and allow mock games to be run in testing."
    - Created GameController interface with declareWinner(), getLogDirectory(), isGameOver(), getWinningPlayerValue() and playerExited().
    - Encapsulated CardGame's player-interaction methods in a gameController instance
- **3b89039 20/10/26** - "feat: TestUtils generate a valid pack file".
    - Created a method to automatically create valid pack files to aid testing.
- **10dd7e9 20/10/26** - "feat: PackTest nested ReadingPackFileTests with before/after setup, validLines and validPackLoadsAllCardsInFileOrder test"
    - Testing the generated pack files.
- **fc61ec1 20/10/26** - “feat: huge rework of locking to follow resource ordering to prevent deadlock with varying player counts; also extracted logging logic for readability.”
    - Upgraded deck locking mechanism to utilise resource reordering to prevent deadlocks.